TAREA 02: Listas, Expresiones Regulares, Pilas y Colas en Python

Manejo de Datos

Facultad de Ciencias

Equipo: Los Bonitos

Integrantes:

De la Peña Driottz María Alejandra

Flores Sánchez Itzel Jessica

León Luna Alan Gaell

Medina Cortés Fernanda

Profesora: Jessica Santizo Galicia
Ayudante: Brenda Lizeth Ramírez Arista

5. Análisis y reflexión

1. Una tabla de complejidades con el número y el nombre de cada problema, la estructura principal que usaste y sus complejidades temporal y espacial en notación Big-O.

Problema	Estructura	Complejidades temporal Big-O	Complejidad espacial Big-O
1.1 #1 Two Sum	Lista y diccionario (dict o tabla hash)	O(n)	O(n)
1.2 #238 Product of Array Except Self	Lista y variables a.	O(n)	O(n)
Extra: #56 Merge Intervals	Lista de intervalos	O(n log n)	O(n)
#65 Valid Number	Expresión Regular	O(n)	O(1)
#592 Fraction Addition and Subtraction	Expresión regular y enteros	O(n)	O(n)
#929 Unique Email Addresses	Expresiones regulares y conjunto (set)	O(nxm)	O(nxm)
#20 Valid Parentheses	Pila (list)	O(n)	O(n)
#150 Evaluate Reverse Polish Notation	Pila de enteros	O(n)	O(n)
#739 Daily Temperatures	Pila monótona	O(n)	O(n)
#232 Implement Queue using Stacks	Dos pilas (listas de Python)	push O(1); pop y peek O(1) amortizado	O(n)
#622 Design Circular Queue	Arreglo de tamaño fijo con índice de cabeza y contador	O(1) en todas las operaciones	O(k)
#239 Sliding Window Maximum	collections.deque de índices (cola monótona)	O(n)	O(k)

2. Una reflexión de entre 300 y 500 palabras que responda:

• ¿Por qué list.pop(0) es O(n) y deque.popleft() es O(1)? Explícalo con base en cómo
Se almacenan los elementos en memoria.
• ¿En qué problema les costó más trabajo elegir la estructura correcta y por qué. Discutan
entre sus compañeros de equipo .
• ¿Qué ventaja tiene re.fullmatch sobre re.match para validar formatos? Da un ejemplo
en el que den resultados distintos.

La diferencia entre list.pop(0) y deque.popleft() se debe en gran parte en que se almacenan los datos en la memoria.
Es decir, las listas de python utilizan arreglos dinámicos, donde los elementos se almacenan en posiciones consecutivas. Al momento de ejecutar list.pop(0), eliminamos el primer elemento y los demás se desplazan una posición para que se conserve el orden, Por ello su complejidad temporal es O(n), ya que el trabajo aumenta según la cantidad de elementos. En cambio, deque. del módulo de collections, está diseñada para insertar y eliminar elementos eficientemente en ambos extremos, por ello su implementación utiliza bloques de memoria, lo que permite eliminar el primer elemento sin desplazar los demás, por eso, deque.popleft tiene su complejidad temporal O(1).
Por lo tanto deque es más eficiente para implementar colas, mientras que las listas son útiles cuando se necesita acceder a ciertos elementos mediante indices.

Discutiendo en equipo sobre cuál problema nos costó más trabajo, coincidimos en que el más complicado para elegir la estructura fue el de **Fraction Addition and Subtraction (#592)**. Al inicio no sabíamos si nos convenía usar listas normales, pilas o intentar separar todo manualmente con ciclos, porque las fracciones venían con signos de más y menos revueltos y denominadores diferentes. Al final nos dimos cuenta de que desglosar la cadena usando expresiones regulares (re.findall) nos facilitaba mucho la vida para extraer los números con todo y su signo, y ya después pudimos manejar la suma de fracciones con aritmética básica. Fue un buen reto porque tuvimos que pensar bien qué herramienta nos ahorraba más código.

Por otro lado, respecto al manejo de expresiones regulares, la principal ventaja de usar **re.fullmatch** sobre **re.match** para validar formatos es que fullmatch revisa que **toda** la cadena cumpla con la regla de inicio a fin. En cambio, re.match solo se fija en que el **comienzo** de la cadena cumpla el patrón, ignorando si al final hay caracteres extra o basura que no deberían estar ahí.

Un ejemplo sencillo donde dan resultados diferentes es si queremos validar un correo o un número y probamos la cadena "12345abc" con el patrón de puros números r'\d+':

- Si usamos re.match(r'\d+', "12345abc"), nos va a decir que **sí coincide** porque se queda solo con el "12345" del inicio e ignora el resto.
- En cambio, si usamos re.fullmatch(r'\d+', "12345abc"), nos regresa None (falla) porque detecta que la cadena no es 100% de puros dígitos debido a las letras "abc" del final. Por eso fullmatch es mucho más seguro para validar entradas completas.